PATHFINDER AI

Project Documentation: Requirements and Technical Reference

Team Nexus
Version 3.0 (final) | 1 to 20 November 2026
Tutor: Mr Satar Sarafa

Product definition, requirements, architecture, data model, AI rules and MVP delivery plan. All technical decisions in this document are final and binding.

1. Purpose and rules for the builder

This document defines what PathFinder AI is and how it is built. It is the source of truth for the Kiro builder agent and the Kiro review agent. Product scope must not be changed silently. If something is missing, contradictory or not possible, the agent must say so and ask the team for a decision instead of inventing an answer.

Final technical decisions

Area	Decision
Architecture	Modular monolith: one codebase and one deployment, split inside into feature modules
Background work	None in the MVP. No worker, queue or Redis
Language and framework	TypeScript strict mode; Next.js App Router
Database	PostgreSQL hosted on Supabase
Authentication	Supabase Auth
ORM and migrations	Drizzle ORM with Drizzle Kit
UI	Tailwind CSS with shadcn/ui
Validation	Zod for API inputs and AI output
AI provider	DeepSeek only, through the Vercel AI SDK and @ai-sdk/deepseek. No multi-provider routing
DeepSeek model	deepseek-v4-flash in non-thinking mode. Not a reasoning model
Package manager	npm; package-lock.json is committed
Hosting	Vercel
Eligibility	Decided by plain application code for structured criteria. The AI cannot decide or override it
Opportunity data	Admin-curated and verified. No crawler
Applications	Users apply on the official provider site. PathFinder never submits applications
Audience	Nigerian undergraduates and recent graduates; global opportunities where the stated eligibility supports them

Exact package versions are installed as the latest stable compatible versions on day one, then fixed by package-lock.json. The first build task checks that deepseek-v4-flash works with the installed ai and @ai-sdk/deepseek versions for structured output. If it does not, the AI module uses text generation with Zod validation.

2. Product overview

PathFinder is an AI-assisted opportunity discovery and career-readiness platform for Nigerian university undergraduates and recent graduates. It helps them find scholarships, SIWES/IT placements, internships, fellowships, graduate programmes, funded training and leadership opportunities around the world where applicants with their nationality and profile may be eligible.
Visitors can browse without an account. Registered users build a profile, see their eligibility, understand missing information, get a readiness plan, save opportunities and track application progress. It is an opportunity recommendation system, not a general-purpose chatbot.

Target users

- Undergraduates: scholarships, SIWES/industrial training, internships, research placements, skills programmes.

- Recent graduates: fellowships, graduate programmes, internships, funded training, further study. NYSC status can affect eligibility and is captured where relevant.
- A programme being international does not mean it accepts Nigerians. Applicant nationality, applicant residence and opportunity destination are three separate data fields.
- Funding types: fully funded, partially funded, stipend, paid, unpaid, tuition-free, travel-supported, accommodation-supported, self-funded.

Main user journey

Step	Behaviour
1. Discover	Visitor browses, searches and filters public opportunities
2. Inspect	Visitor reads provider, eligibility, funding, deadline, source and application link
3. Register	User creates an account and fills in a profile
4. Assess	System checks structured eligibility criteria and calculates relevance separately
5. Understand	User sees met, not met, unknown and needs-review criteria
6. Prepare	On request, AI explains the computed results and builds a readiness plan from verified facts
7. Track	User saves the opportunity and records application progress
8. Apply	User follows the official provider link

Public browsing needs no account. Personal assessment, saved opportunities and tracking need an account. Profile onboarding collects only what supports eligibility, relevance or preparation: education status, institution, course, academic level, graduation year, nationality, residence, NYSC status where applicable, skills, experience and career goals. Application statuses are: Not started, In progress, Submitted, Accepted, Rejected, Withdrawn. PathFinder never claims an application was submitted unless the user records that status.

3. Eligibility, relevance and readiness

Operation	Question	How it works
Eligibility	Does the user meet the stated mandatory conditions?	Plain code on structured criteria; explicit unknown or review states otherwise
Relevance	How well does the opportunity fit the profile and goals?	Transparent ranking using approved profile and opportunity attributes
Readiness	What can the user improve?	AI guidance based on verified requirements and known profile gaps

Eligibility outcomes

Outcome	Meaning
Eligible	All known mandatory conditions are met and nothing required is unresolved
Likely eligible	Known conditions look met, but important details are unconfirmed
Needs review	Criteria or profile data are unclear or not enough for a dependable result
Not eligible	At least one known mandatory condition is not met

- Missing information is unknown, never a pass.
- A known mandatory failure must never be hidden by a high relevance score.
- A relevance score is not an eligibility probability and does not predict selection, admission or funding.
- Each criterion has a type, operator, required value, mandatory or preferred flag and a source reference where possible.

- Each criterion returns met, not met, unknown or needs review, with a short reason.
- A known mandatory failure gives Not eligible. Unknown mandatory information blocks a definite Eligible.
- Natural-language criteria that cannot be safely structured stay flagged for review. Do not invent an interpretation.
- Relevance uses education, skills, experience, interests and goals. Factors must be understandable; no arbitrary weights without a reason and tests.
- Readiness tasks must come from verified requirements or be labelled as general advice. Users can review, edit and mark tasks complete.

4. AI behaviour (DeepSeek)

All LLM calls go through the Vercel AI SDK using @ai-sdk/deepseek, behind one small internal AI module (src/lib/ai). Use streamText for streamed text and generateObject for structured output where the installed versions support it. This module is a maintainability boundary, not a multi-provider framework.

Allowed

- Explain already-computed eligibility results.
- Explain relevance factors without presenting them as selection chances.
- Generate structured readiness plans from verified opportunity facts and the relevant profile data.
- Help interpret ambiguous criteria, while leaving them flagged for human review.

Prohibited

- Deciding, calculating, overriding or certifying eligibility.
- Inventing requirements, nationality or residence rules, age limits, deadlines, funding, stipends, links, documents or provider decisions.
- Turning missing information into a positive claim.
- Promising admission, jobs, awards, visas or selection.
- Exposing one user's private data to another user.

Grounding, validation and failure

Send only relevant verified facts, source context, computed eligibility results and the minimum profile attributes. Validate structured output with Zod before showing or saving it. Reject or flag malformed output and unsupported claims. Keep source references and timestamps. If DeepSeek is unavailable, public browsing and eligibility still work and the AI feature shows a clear error with no made-up fallback. API keys stay on the server.

5. Opportunity curation and verification

The MVP uses manual admin curation. Admins capture structured details, check official sources, record evidence, and publish only after review. The data team (Samuel and Efeteyan) collects records from official sources for the admin to enter.

Data	What to keep
Identity	Title, provider, category, official programme page
Eligibility	Nationality, residence, education level, field, age, experience, language, documents, other stated criteria
Location	Destination, city, remote / in-person / hybrid
Timing	Opening date, deadline, timezone, programme dates, rolling or no deadline
Funding	Each component, amount, currency, frequency, evidence state

Data	What to keep
Internship	SIWES/IT relevance, duration, work arrangement, paid status, academic level
Verification	Source URL, last checked date, reviewer, outcome, notes, lifecycle status

Unknown-data rules

- Do not infer Nigerian eligibility from an international brand or host.
- Do not infer funding from a programme title.
- Do not infer an open deadline from an old listing.
- A page that loads does not prove its details are current.
- Where evidence is missing, conflicting or unclear, keep the uncertainty and write unknown.
- Prefer official provider sources when facts conflict.
- Admins recheck records during curation and record a last-verified date. No universal fixed recheck interval is claimed.

Lifecycle states

State	Meaning
Draft	Captured, not reviewed, not public
Under review	Being checked against its source
Published / open	Approved for public display; source supports open status
Upcoming	Future cycle confirmed, applications not yet open
Closing soon	Open and near its stated deadline
Closed / expired	Window ended or source confirms closure
Unknown / needs recheck	Current status cannot be established reliably
Archived	Removed from active discovery; kept for history

6. Data model (entity inventory)

This is an entity inventory, not a final migration. The builder agent proposes a normalised schema in Drizzle, with keys, constraints, indexes and deletion behaviour, and stops for approval from Joshua and Moses before any migration runs. Do not force every concept into its own table, and do not impose an arbitrary table limit.

Entity	Purpose
users	Application identity linked to the Supabase Auth user id, display name, timestamps
profiles	User type, nationality, residence, education status, institution, course, level, graduation year, NYSC status, goals
skills / user_skills	Skills and their link to profiles
experiences	Projects, work, volunteering, research
organizations	Provider or host organisation, website, country
opportunities	Title, description, category, lifecycle status, delivery mode, official URL, deadline, verification state
opportunity_categories	Category list
opportunity_locations	Destination and delivery location, separate from applicant nationality and residence
applicant_eligibility	Nationality and residence restrictions and notes
eligibility_criteria	Criterion type, operator, required value, mandatory or preferred, source reference

Entity	Purpose
funding_components	Tuition, stipend, salary, accommodation, travel, fees; amount, currency, frequency, evidence state
opportunity_sources	Source URLs, source type, checked date, evidence notes
verification_events	Reviewer, date, outcome, notes
eligibility_assessments	User and opportunity, status, per-criterion results, timestamp
match_results	Relevance score and factors; not an eligibility probability
saved_opportunities	Bookmarks
applications	Status, applied date, private notes
action_plans / tasks	Readiness plan and its tasks

Schema review gate

- Present the table list and the reason for each.
- Give primary and foreign keys, unique and check constraints, and indexes.
- Explain relationships, ownership and admin access.
- Explain how unknown values, funding components and geographic eligibility are stored.
- Define deletion, archive and audit behaviour.
- Say what was combined or left out, and why.
- Wait for approval from Joshua and Moses before creating migrations.

7. System architecture

One codebase and one deployment on Vercel, split into feature modules, with a shared Supabase PostgreSQL database. Page components show data; domain logic stays in modules. Modules talk through typed interfaces.

Module	Responsibility
Identity	Supabase Auth, sessions, account
Profiles	Onboarding, education, skills, experience
Opportunities	Catalogue, details, categories, funding, locations, search
Eligibility	Rule-based criteria evaluation and criterion-level results
Matching	Relevance calculation, ranking factors, explanations
Readiness	DeepSeek explanations and structured readiness plans
Applications	Saved opportunities and application progress
Verification / Admin	Review, publishing, source records, audit events

Request flow: Browser, Next.js app, feature module(s), Supabase PostgreSQL. AI requests also go through the internal AI module, the Vercel AI SDK, then DeepSeek.

Excluded from the MVP

- Microservices and multi-provider AI routing.
- Separate workers, Redis, BullMQ.
- Vector database, pgvector, advanced RAG.
- CV or document upload and parsing.
- Large-scale crawling, automatic application submission, WhatsApp or email reminders, CV or cover-letter generation, AI interview coaching, employer dashboards, university integrations, multi-agent orchestration.

8. Project folder structure

Starting structure. The builder agent checks it against the installed Next.js version before creating files.

```
pathfinder/
|-- .kiro/
|   |-- steering/        # persistent project guidance
|   `-- specs/           # feature requirements, design, tasks
|-- docs/                # product, architecture, data notes
|-- public/
|-- src/
|   |-- app/
|   |   |-- (public)/opportunities/
|   |   |-- (auth)/sign-in/
|   |   |-- (auth)/sign-up/
|   |   |-- dashboard/
|   |   |-- admin/
|   |   `-- api/         # only where route handlers are needed
|   |-- modules/
|   |   |-- identity/
|   |   |-- profiles/
|   |   |-- opportunities/
|   |   |-- eligibility/
|   |   |-- matching/
|   |   |-- readiness/
|   |   |-- applications/
|   |   `-- verification/
|   |-- db/
|   |   |-- schema/
|   |   |-- migrations/
|   |   `-- index.ts
|   |-- lib/
|   |   |-- ai/          # DeepSeek module and schemas
|   |   |-- auth/
|   |   |-- validation/
|   |   `-- utils/
|   |-- components/      # shared UI only, no domain logic
|   `-- tests/           # unit, integration, e2e
|-- .env.example
|-- AGENTS.md
|-- REVIEW.md
|-- drizzle.config.ts
|-- next.config.ts
|-- package.json
`-- README.md
```

- Domain logic lives in feature modules, not page components.
- Eligibility must be testable without the UI or an AI call.
- The AI module takes typed input and returns validated structured output.
- Use one documented schema and migration path.

9. Functional requirements

Identifiers are fixed. Do not renumber or merge them.

ID	Requirement
FR-01	Visitors must be able to browse published opportunities without creating an account.
FR-02	Visitors must be able to search and filter opportunities using supported structured attributes.
FR-03	The system must provide opportunity detail pages with source and application links.
FR-04	Users must be able to register, authenticate and manage their account.
FR-05	Users must be able to create and update a profile appropriate to undergraduate or recent-graduate status.
FR-06	Profiles must support relevant education, nationality, residence, skills, experience, career goals and NYSC status where applicable.
FR-07	The catalogue must support scholarships, SIWES/IT, professional internships, fellowships, graduate programmes, funded training and leadership opportunities.
FR-08	The system must distinguish opportunity destination from eligible applicant nationality and residence.
FR-09	The system must represent funding components independently and allow unknown or not-stated values.
FR-10	Opportunity records must store official source information, lifecycle status and last verification timestamp.
FR-11	Authorized administrators must be able to create, edit, review, publish, close and archive opportunities.
FR-12	Eligibility criteria must be stored in a structured form where feasible, including mandatory or optional status.
FR-13	The eligibility engine must compare profile data against known mandatory criteria.
FR-14	The system must distinguish eligible, likely eligible, needs review and not eligible outcomes.
FR-15	The system must show criterion-level explanations and distinguish unmet requirements from unknown data.
FR-16	The matching engine must calculate relevance separately from eligibility.
FR-17	The system must explain the main factors contributing to a recommendation.
FR-18	Users must be able to request AI-assisted explanations and readiness plans.
FR-19	AI outputs must be validated and must not invent opportunity requirements, deadlines or funding facts.
FR-20	Users must be able to save and unsave opportunities.
FR-21	Users must be able to record and update application progress.
FR-22	The system must link users to the official provider application channel.
FR-23	The system must prevent expired or unverified listings from being presented as confirmed open opportunities.
FR-24	The system must allow users to report an inaccurate or expired opportunity. Lowest priority: build last, only if time allows.

10. Technical requirements

ID	Requirement
TR-01	Implement the application in Next.js with TypeScript strict mode.
TR-02	Use PostgreSQL, hosted on Supabase, as the primary relational data store.
TR-03	Use Drizzle ORM and version-controlled database migrations.
TR-04	Validate API inputs and structured AI outputs at runtime.
TR-05	Enforce authorization on the server for every protected operation; hiding UI controls is not authorization.

ID	Requirement
TR-06	Keep user profile, assessment and application data private to the owning user and authorized administrators.
TR-07	Implement eligibility checks in deterministic application logic wherever criteria are structured.
TR-08	Keep eligibility, relevance ranking and readiness analysis as separate domain operations.
TR-09	Route LLM calls through an internal service boundary and validate outputs against schemas; use DeepSeek as the sole provider through the Vercel AI SDK.
TR-10	Store source provenance and verification timestamps for published opportunity facts.
TR-11	Represent missing, unknown and ambiguous facts explicitly.
TR-12	Use database constraints and indexes for integrity and common catalogue queries.
TR-13	Keep secrets and provider API keys on the server; never expose them in client bundles.
TR-14	Use environment variables and provide an example environment file without real secrets.
TR-15	Maintain automated tests for eligibility rules, authorization boundaries and critical workflows.
TR-16	Use accessible, responsive UI patterns for mobile and desktop.
TR-17	Record timestamps in a consistent timezone strategy and render deadlines in a clearly identified timezone.
TR-18	Do not introduce vector search, Redis queues or separate services without a documented need.

11. Non-functional expectations

Area	Expectation
Security	Secure sign-in, server-side authorization, protected secrets, HTTPS, least access
Privacy	Collect only profile data the features need; give users account and data management
Accuracy	Eligibility results trace to criteria and profile facts; unknown never becomes confirmed
AI safety	No made-up eligibility, deadlines, funding, provider decisions or guarantees
Freshness	Published records show the last-verified date and are not labelled open after confirmed closure
Performance	Target: normal catalogue search within 2 seconds under normal MVP load. Measure it; do not claim it as a guarantee
Availability	Catalogue and eligibility stay usable if DeepSeek is down
Usability	Discovery and detail pages work on common phone and desktop screens
Maintainability	Typed boundaries, modular code, documented setup, automated tests
Explainability	Show the factors behind eligibility and relevance; no opaque certainty

12. Delivery plan (1 to 20 November 2026)

Phase	Output	Dates
1. Planning	Idea approved, scope, stack and documents final	01/11 to 09/11
2. Foundation	Project setup, Supabase, Drizzle migrations, sign-in	10/11 to 11/11
3. Core workflows	Profiles, catalogue, admin area	11/11 to 14/11
4. Eligibility and AI	Eligibility engine, relevance ranking, AI readiness plans	14/11 to 16/11
5. Tracking and testing	Saved opportunities, application tracking, tests, fixes	16/11 to 18/11
6. Finish	Final documentation, demo script, presentation, submission	18/11 to 20/11

MVP includes: public catalogue, profile onboarding, curated opportunity database, search and filters, admin verification, eligibility checks, relevance ranking, AI readiness plans, saved opportunities and basic application tracking.
Deferred: crawling, CV upload or parsing, reminders, automatic submission, CV or cover-letter generation, interview coaching, employer dashboards, university integrations, multi-agent orchestration, advanced RAG, separate worker infrastructure.

13. Team and responsibilities

Name	Role	Responsibility
Joshua Samson Anuoluwapo	Team Leader (Technical)	Coordination; product requirements; frontend pages and user flows; eligibility and relevance display; testing; reviews and pushes to the repository
Moses Uvie Uvietabor	Team Member (Technical)	Database schema and migrations; sign-in and server-side access control; backend and API; DeepSeek and AI SDK connection; reviews and pushes to the repository
Samuel Oladunjoye Chigozirim	Team Member	Opportunity data collection from official sources; source-link checking
Efeteyan Etinosa Miracle	Team Member	Opportunity data collection; cross-checking records
Oluwasola Josiah Temidayo	Team Member	Project documentation; workflow descriptions
Shittu Abdulmalik Olasunkanmi	Team Member	Pitch and presentation; problem and value explanation
Adebayo Isaac	Team Member	Demo preparation and plain-language walkthrough

Joshua and Moses approve the Kiro builder's work, check each other's changes, and commit and push to the shared GitHub repository when each piece is done. The data team uses official sources only, records the source link and the date checked, writes unknown when a detail is missing, and aims for about twenty verified opportunities before the demo.

14. Risks and checks

Risk	Handling
Opportunity freshness	Manual curation; last-verified date on every record
Global eligibility complexity	Keep nationality, residence and destination separate; use review states
Ambiguous criteria	Keep source evidence; require review when a rule cannot be safely structured
Matching weights	Use transparent factors; do not tune arbitrarily
DeepSeek quality, cost, speed	Test early; non-thinking flash model keeps cost and delay low; clear errors on failure
Structured output support	Verified on the first build task; fallback is text generation with Zod validation
Funding is multi-part	Store each component with its evidence state
Deadlines and timezones	Keep timezone context; model repeating cycles where needed

15. Agent workflow

- The builder agent follows AGENTS.md and the build order in it, one small task at a time.
- Before each task, it states what it will create or change and waits for approval from Joshua or Moses.
- After each task, it gives a completion report and stops.
- The review agent (REVIEW.md) then checks the work against this document, without changing code.
- Fixes are made, then the next task starts.

- The database schema and any migration always need approval first.
- Do not start feature work until requirements, architecture, data model and the first task are approved.

End of PathFinder AI Project Documentation